

Project Analysis: ClaimSwift – Vehicle Insurance Claims Platform

Overview: The ClaimSwift project aims to build a cloud-native insurance claims processing platform that streamlines end-to-end claims workflows. Its core goal is to allow policyholders to submit claims (with photos/documents), track status in real time, and retrieve settlement documents, while giving claims adjusters and managers tools to assess, approve/disapprove, and manage claims efficiently. The platform is explicitly designed as a microservices-based system (using Spring Boot/Cloud and Docker) with separate services for submission, document processing, assessment, payment, notifications, reporting, etc. This modular approach (common in modern insurance IT) lets the system break the complex claims process into smaller, specialized tasks – for example, dividing claim processing into document upload, fraud detection, payment handling and validation services ¹. By containerizing each service (via Docker), the team can ensure consistent environments from development through production ².

Goals & Scope: The primary goals are to **(a)** automate and accelerate the claims lifecycle (from first notice of loss to settlement), **(b)** provide real-time updates and dashboards for each user role, and **(c)** meet regulatory/security requirements (audit trails, encryption, role-based access). Scope includes implementing all major features identified in the user stories: secure login/logout, claim submission with file upload, status tracking, history viewing, claims adjuster dashboard (approve/adjust/disapprove/assign claims), and management functions (assigning adjusters, managing adjuster availability, reporting/analytics). In practice, the scope covers backend microservices (one per major function), a responsive AngularJS front end (policyholder portal, adjuster workspace, manager dashboard), and infrastructure components (API gateway, service registry, CI/CD pipeline). Success criteria would be defined in measurable terms – for example, delivering all user-story features on schedule, with acceptable performance and security. As a general rule, project success means completing the system *on time, within budget, and according to scope*, while satisfying end-user needs ³.

Stakeholders & Success Criteria: Key stakeholders include the **policyholders, claims adjusters, and insurance managers** who will use the system, as well as the project sponsor/owner and development team. Their needs drive the features: policyholders need a simple claims portal; adjusters need efficient work queues and decision tools; managers need oversight dashboards and reporting. Additional stakeholders include IT/DevOps teams (infrastructure, security) and potentially external partners (e.g. payment gateways). In Agile fashion, maintaining clear communication with stakeholders at each sprint is critical – studies note that effective stakeholder engagement and trust directly improve project success in agile projects ⁴. Success criteria should be agreed with stakeholders early on – for example, “policyholders can submit a claim and receive status updates within seconds,” or “the system handles N claims per hour under load.” In general, we would gauge success by meeting time/budget targets *and* fulfilling the agreed feature set and quality targets: i.e. system functions correctly according to the user stories, without major bugs, and stakeholders report being satisfied with the delivered functionality ³ ⁴.

Timeline & Milestones: The project is organized into three Agile sprints. In **Sprint I**, the team designs the overall microservices architecture and builds the foundation: user authentication (with multi-factor

security), the AngularJS policyholder portal, the claim submission service with file validation, and basic document upload/storage. Infrastructure setup (Docker, API gateway, Eureka service registry) is also completed. **Sprint II** adds the core processing engine: the claims assessment service, adjuster dashboard and assignment logic, approval/disapproval flows, and a real-time notification system. Spring Security with hierarchical roles is enforced. **Sprint III** focuses on monitoring, management and completion: a reporting & analytics service, manager dashboard (with BI charts), payment processing integration, and compliance/audit logging. At the end of Sprint III the entire system is integrated and deployed (locally or in cloud containers). This stepwise approach – building infrastructure and core features first, then adding advanced capabilities – follows Agile best practices for incremental delivery. Each sprint would typically last a few weeks, with demos at each sprint end. (Success would be measured by completion of the listed sprint goals on schedule ³.)

Budget & Resources: The project uses primarily open-source technologies (Spring Boot, PostgreSQL, Docker, Jenkins), so software licensing costs are minimal. Hardware/infrastructure could be on-premises servers or cloud VMs/containers – assume some budget for servers or cloud hosting. Key human resources include Java/Spring back-end developers, a front-end Angular developer, a DevOps engineer (for containers/CI), a quality assurance/test engineer, and possibly a UI/UX designer. Specialized skills in microservices, API design, and security are important. Testing tools (JUnit, Mockito, Postman) are standard and open-source. Since this is a capstone, budget is likely limited, so the team should optimize for free tools and efficient development (microservices allow scaling only parts of the system, which can save resources ⁵). However, microservices projects can demand more skill and coordination, so plan for enough experienced personnel.

Technical Approach: The architecture is **cloud-native, microservices-based**. Each major function – claim submission, document processing, claim assessment, payment, notification, reporting – runs as its own Spring Boot service, registered via Eureka and exposed through an API gateway. Databases are segregated by domain (one for user data, one for claims, one for documents, etc.) to enforce microservices autonomy. The front end uses AngularJS (plus Bootstrap/HTML5) to build responsive UIs for each role. Services communicate via REST APIs and optionally event messaging (RabbitMQ or WebSockets) for real-time updates. All services are packaged in Docker containers to ensure consistent deployment environments ². Jenkins (or similar) pipelines would build, test and deploy each container. This approach matches industry patterns: cloud-native microservice architectures provide scalability and agility for insurance platforms ⁶. For example, a DZone analysis of insurance systems notes that a “scalable, modular, and cloud-native microservice architecture” is ideal to enable rapid analytics and flexibility across the enterprise ⁶. In practice, this means using Spring Security on each service for role-based access, and implementing failover/retry patterns (sagas or circuit breakers) to handle distributed transactions. An API gateway can enforce rate limiting and serve as a façade for authentication.

Deliverables & Quality Assurance: Expected deliverables include: (1) source code and deployed instances of all microservices (submission, processing, assessment, payment, notification, reporting), (2) AngularJS front-end applications for policyholders, adjusters, and managers, (3) documentation (architecture diagrams, API specs, deployment guides), and (4) a suite of automated tests. Quality will be ensured through multiple strategies. The team will write **unit tests** and **service-level tests** for each component (using JUnit and Mockito, as is common for Java microservices ⁷). Integration and end-to-end tests will verify that services work together (using tools like Postman for API tests and possibly contract testing to prevent API mismatches). The plan already mentions using JUnit, Mockito and Postman – industry best practices encourage building such automated tests and integrating them into CI/CD pipelines ⁷ ⁸. For example, including testing in the Jenkins build pipeline ensures new code is verified before deployment ⁸.

Additional quality measures include code reviews, static analysis, and performance/load testing. Because insurance involves sensitive data, strict security reviews (penetration testing, code scanning for vulnerabilities) should be done. The system will maintain audit logs of all claim actions for compliance. Finally, non-functional quality: each service should meet defined performance benchmarks (e.g. handle N submissions per minute) and recovery needs (auto-restart containers on failure).

Risks & Mitigation: Several risks are inherent in this project. **Data consistency** is a key challenge in microservices: since each service has its own database, keeping data in sync (e.g. when a claim is approved and a payment is triggered) is complex. We must use appropriate patterns (such as transactional sagas or event sourcing) to avoid inconsistent state ⁹. If not handled, customers might see conflicting information (a classic microservices pitfall) and lose trust ⁹. **Security & Compliance** is another risk: handling personal data and financial transactions requires strong encryption (in transit and at rest) and careful access controls. The architecture must enforce strict role-based access and possibly multi-factor auth (as the plan notes). The **complexity of distributed systems** is also a risk: more services means more points of failure and integration difficulties. For instance, “chatty” inter-service calls can cause latency or bottlenecks, and insufficient logging can lead to the dreaded “black box” syndrome where outages are hard to diagnose ¹⁰. To mitigate this, we will implement centralized logging/monitoring (using ELK stack or similar) so we have unified metrics and traces ¹⁰. **Schedule risk** exists if features prove harder than expected; tight coordination is needed to avoid lagging sprints. The team should regularly reassess scope and prioritize core features if needed. Finally, **resource risk**: the technology stack (Spring Cloud, Docker, etc.) requires specialized skills. We assume the developers are already familiar with these tools. Training or external expertise may be needed to get up to speed, which should be planned early.

Recommendations: Given the above, we recommend the team follow best practices for Agile microservices projects. First, **incremental delivery and stakeholder review** – at each sprint end, review demos with actual users (adjusters, policyholders) to validate assumptions. Maintain open communication with all stakeholders (product owner, sponsor, end users), as effective communication greatly increases chances of success ⁴. Second, **test early and often**: adopt “shift-left” testing by writing unit and integration tests as code is developed ¹¹ ⁸. Integrate these into a CI/CD pipeline (e.g. Jenkins) so that each new build is automatically tested before deployment. Third, **observability**: implement comprehensive logging, metrics and distributed tracing from the start. The team should treat monitoring as a priority, because lack of visibility is a common failure mode in microservices ¹⁰. Fourth, **security and compliance**: use the API gateway and service-level security to enforce data access rules, and regularly audit access logs. The plan’s focus on encryption, MFA, and audit trails is correct – ensure these are tested thoroughly. Finally, **scalability and resilience testing**: simulate peak loads (e.g. many policyholders submitting claims) to verify that services scale independently. Because each service can be deployed in Docker, the team can autoscale hot services and avoid over-provisioning, improving cost efficiency ⁵ ².

In summary, ClaimSwift’s design is well-aligned with modern patterns for insurance software: a microservices architecture provides scalability and flexibility ⁵ ⁶. The project success will hinge on disciplined project management (clear milestones, stakeholder demos, budget tracking) and technical rigor (automated testing, security reviews, monitoring). With careful attention to these factors, the team can deliver a robust claims platform that meets user needs and is maintainable in the long run.

Sources: Industry analyses of microservices in insurance ¹ ⁶, Agile project management guidelines ⁴ ³, and microservices best practices ² ⁷ ¹⁰ informed this assessment.

1 5 How Microservices Architecture is Revolutionizing Insurance Software Development - Mobile App Development Company in South Africa, USA, India & Globally

<https://depextechnologies.com/blog/how-microservices-architecture-is-revolutionizing-insurance-software-development/>

2 Dockerizing Microservices: Best Practices and Patterns | by Ankita Patel | Medium

<https://medium.com/@FullStackSoftwareDeveloper/dockerizing-microservices-best-practices-and-patterns-610c281393b2>

3 11 Project Success Criteria Examples | Motion | Motion

<https://www.usemotion.com/blog/success-criteria-examples>

4 The Role of Stakeholders in Agile Project Management - isixsigma.com

<https://www.isixsigma.com/agile/the-role-of-stakeholders-in-agile-project-management/>

6 Cloud-Native Microservice Architectures for Insurance

<https://dzone.com/articles/cloud-native-microservices-architectures-insurance>

7 8 11 QA in Microservices Architecture - Best Practices and Challenges

<https://www.pixelqa.com/blog/post/qa-in-microservices-architecture-best-practices-and-challenges>

9 10 The Hidden Risks Inside Modern Microservices: A Leadership Guide to Building Reliable, Scalable Systems

<https://aimconsulting.com/insights/microservices-hidden-risks-leadership-guide/>